

Descripción **PlayerController**

Nuestro script **PlayerController** es un componente de Unity, el cual fue diseñado para controlar el movimiento en primera persona de un jugador utilizando un **CharacterController**. Además del movimiento básico (caminar, correr, saltar y agacharse), incluye dos mecánicas avanzadas de movilidad: **un sistema de Dash (un impulso rápido)** y **un sistema de Gancho**, ambos vinculados a elementos visuales de la interfaz de usuario (UI) para mostrar sus tiempos de recarga (**cooldowns**).

¿Qué componentes requerimos?

Componentes Requeridos

- **CharacterController:**

El script obligó a que el GameObject tenga este componente (`[RequireComponent(typeof(CharacterController))]`), el cual se utiliza para gestionar las colisiones y el movimiento físico del jugador sin depender de fuerzas rígidas (**Rigidbody**).

Estructura de Variables

1. Referencias

Primero tomamos las referencias de lo que necesitamos para que nuestros controladores funcionen correctamente y se almacenen donde deberían, dependiendo de nuestras necesidades, cambiamos el tipo de acceso de **-publico-** a **-privado-** a ciertos sectores.

playerCamera (pública): Cámara de la escena que representa la vista del jugador.

cameraTransform (privada): Almacena el **Transform** de la cámara para optimizar su manipulación.

characterController (privada): Referencia interna al controlador de físicas.

myTransform (privada): Referencia interna al **Transform** del jugador para mejorar el rendimiento.

2. Movimiento Base y Salto

```
if (!isDashing)
{
    Vector3 forward = myTransform.forward;
    Vector3 right = myTransform.right;

    bool isRunning = Input.GetKey(KeyCode.LeftShift);
    float currentSpeedMultiplier = isRunning ? runSpeed : walkSpeed;

    float currentSpeedX = canMove ? currentSpeedMultiplier * Input.GetAxis("Vertical") : 0;
    float currentSpeedY = canMove ? currentSpeedMultiplier * Input.GetAxis("Horizontal") : 0;

    float movementDirectionY = moveDirection.y;
    moveDirection = (forward * currentSpeedX) + (right * currentSpeedY);

    if (Input.GetButton("Jump") && canMove && characterController.isGrounded)
    {
        moveDirection.y = jumpPower;
    }
    else
    {
        moveDirection.y = movementDirectionY;
    }

    if (!characterController.isGrounded)
    {
        moveDirection.y -= gravity * Time.deltaTime;
    }
}
```

Estas bases dan vida al movimiento de nuestro juego y logramos

walkSpeed (**pública**, def: 12): Velocidad al caminar de cara a los cálculos actuales.

runSpeed (**pública**, def: 24): Velocidad al correr con Shift izquierdo.

jumpPower (**pública**, def: 7): Fuerza del salto hacia arriba.

gravity (**pública**, def: 10): Fuerza de gravedad aplicada verticalmente cuando el jugador no toca el suelo.

baseWalkSpeed / baseRunSpeed (**privadas**): Guardan los valores originales de velocidad para restaurarlos después de agacharse.

3. Cámara

```
if (canMove)
{
    rotationX -= Input.GetAxis("Mouse Y") * lookSpeed;
    rotationX = Mathf.Clamp(rotationX, -lookXLimit, lookXLimit);
}
```

```
        cameraTransform.localRotation = Quaternion.Euler(rotationX, 0, 0);

        myTransform.Rotate(Vector3.up, Input.GetAxis("Mouse X") * lookSpeed);
    }
}
```

El movimiento de cámara es suave y se siente fluido con el personaje, logramos crear ciertos límites para que estos no se rompan en dadas ocasiones.

lookSpeed (**pública**, def: 2): Sensibilidad del ratón.

lookXLimit (**pública**, def: 45): Límite de ángulo (grados) para mirar hacia arriba o hacia abajo, evitando giros de 360 grados.

4. Agacharse

```
if (wantToCrouch && !isCrouching)
{
    characterController.height = crouchHeight;
    walkSpeed = crouchSpeed;
    runSpeed = crouchSpeed;
    isCrouching = true;
}
else if (!wantToCrouch && isCrouching)
{
    characterController.height = defaultHeight;
    walkSpeed = baseWalkSpeed;
    runSpeed = baseRunSpeed;
    isCrouching = false;
}

characterController.Move(moveDirection * Time.deltaTime);
}
```

Estos establecidos podrán crear una experiencia más cercana al suelo, lo cual, nos expande el mundo de posibilidades y de opciones dentro de nuestro mundo.

defaultHeight (**pública**, def: 2): Altura normal del **CharacterController**.

crouchHeight (**pública**, def: 1): Altura del controlador al agacharse.

crouchSpeed (**pública**, def: 3): Velocidad máxima permitida mientras el jugador está agachado.

isCrouching (**privada**): Bandera para controlar el estado actual de agachado y evitar procesamiento innecesarios en las físicas.

5. Mecánica: Dash

```
if (Input.GetKeyDown(KeyCode.Q) && Time.time >= nextDashTime && !isDashing)
{
    dashDirection = new Vector3(
        Input.GetAxisRaw("Horizontal"),
        0,
        Input.GetAxisRaw("Vertical")
    );

    dashDirection = myTransform.TransformDirection(dashDirection);

    if (dashDirection.sqrMagnitude == 0)
    {
        dashDirection = myTransform.forward;
    }

    StartCoroutine(DashRoutine());
}
```

```
IEnumerator DashRoutine()
{
    isDashing = true;

    float startTime = Time.time;

    Vector3 dashVelocity = dashDirection.normalized * dashSpeed;

    while (Time.time < startTime + dashDuration)
    {
        characterController.Move(dashVelocity * Time.deltaTime);
        yield return null;
    }

    isDashing = false;
    nextDashTime = Time.time + dashCooldown;
}
```

Esta mecánica nos brinda una movilidad mas avanzada y nos ayuda mucho con las plataformas.

dashSpeed (pública, def: 30): Velocidad del impulso.

dashDuration (pública, def: 0.2): Duración del trayecto del Dash en segundos.

dashCooldown (pública, def: 1): Tiempo de espera requerido antes de poder usarlo de nuevo.

nextDashTime (privada): Marca de tiempo interna (**Time.time**) para saber cuándo expira el cooldown.

isDashing (privada): Bandera que bloquea el movimiento normal mientras se ejecuta la corrutina del Dash.

6. Mecánica: Gancho (Grappling Hook)

Con estas variables logramos darle vida a una mecánica de movimiento muy frenética, al mero estilo de **ULTRAKILL**.

```
if (Input.GetKeyDown(grappleKey) && !isGrappling && Time.time >= nextGrappleTime)
{
    StartGrapple();
}

if (isGrappling)
{
    MoveToGrapplePoint();
    return;
}
```

```
void StartGrapple()
{
    if (Physics.Raycast(cameraTransform.position, cameraTransform.forward, out RaycastHit hit, maxDistance,
grappleLayerMask))
    {
        grapplePoint = hit.point;
        isGrappling = true;
    }
}

void MoveToGrapplePoint()
{
    Vector3 directionToPoint = grapplePoint - myTransform.position;

    if (directionToPoint.sqrMagnitude < 4f)
    {
        isGrappling = false;
        nextGrappleTime = Time.time + grappleCooldown;
        return;
    }

    characterController.Move(directionToPoint.normalized * grappleSpeed * Time.deltaTime);
}
}
```

maxDistance (pública, def: 50): Rango máximo de alcance para que el gancho impacte una superficie.

grappleSpeed (pública, def: 25): Velocidad a la que el jugador es arrastrado hacia el punto de impacto.

grappleKey (pública, def: **KeyCode.E**): Tecla asignada para disparar el gancho.

grappleLayerMask (pública, def: **Everything**): Capas físicas con las que el gancho puede colisionar.

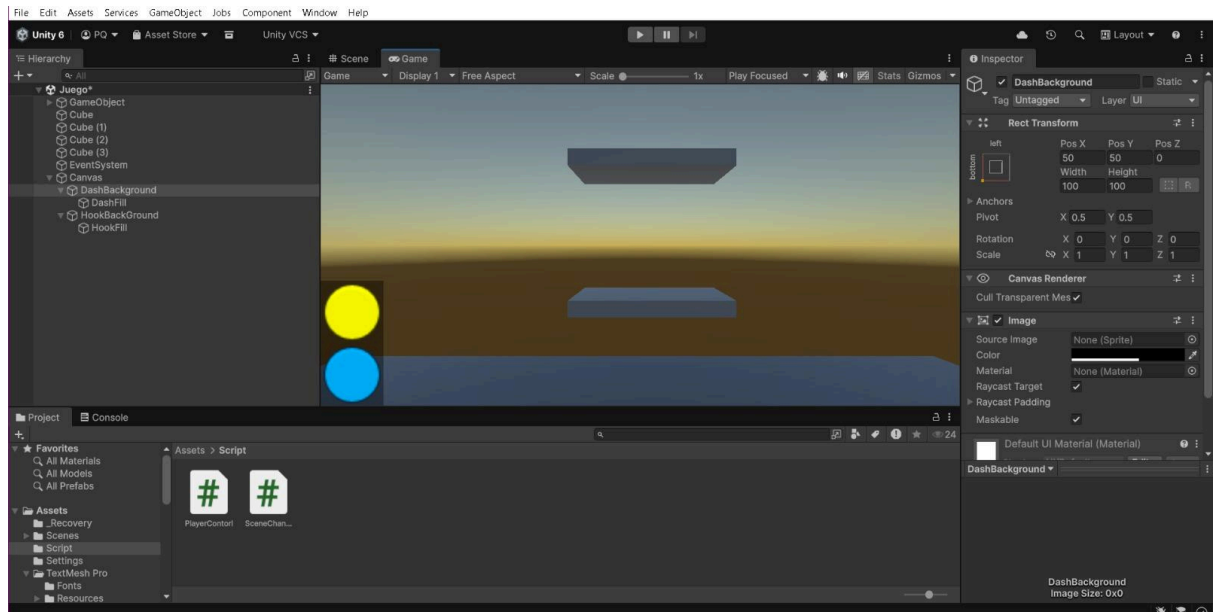
grappleCooldown (pública, def: 2): Tiempo de espera tras soltarse del gancho.

nextGrappleTime / **isGrappling** / **grapplePoint** (privadas): Controladores internos de tiempo, estado y la posición exacta en el espacio donde se enganchó.

6.1 UI: Gancho + Dash

```
if (dashFillImage != null)
{
    if (Time.time < nextDashTime)
    {
        dashFillImage.fillAmount = 1f - ((nextDashTime - Time.time) / dashCooldown);
    }
    else
    {
        dashFillImage.fillAmount = 1f;
    }
}
```

```
if (grappleFillImage != null)
{
    if (Time.time < nextGrappleTime)
    {
        grappleFillImage.fillAmount = 1f - ((nextGrappleTime - Time.time) / grappleCooldown);
    }
    else
    {
        grappleFillImage.fillAmount = 1f;
    }
}
```



Aquí podemos apreciar la UI de los botones que funcionan con el **-Dash-** y **-Gancho-**

Estos reaccionan con los respectivos cooldowns de las propias habilidades que el jugador decida usar.

- **Amarillo** = **Gancho**
- **Azul** = **Dash**

Con las visuales, logramos que estos reaccionaran dependiendo a sus respectivos cooldowns.

dashFillImage (**pública**): Componente **Image** (tipo Filled) en la UI que muestra de forma visual la recarga del Dash.

grappleFillImage (**pública**): Componente **Image** en la UI para la recarga del Gancho.